DCC006/DCC017 Organização de computadores I/Introdução à Arquitetura de Computadores

Trabalho Prático #2

Professor: Daniel Fernandes Macedo e Omar Paranaiba Vilela Neto

Antes de começar seu trabalho, leia todas as instruções abaixo.

1. Instruções gerais

- O objetivo desse trabalho é se familiarizar com a Linguagem de Descrição de Hardware Verilog. Será disponibilizado no moodle um arquivo .ipynb com uma implementação do RISCV 5 estágios em Verilog. Sua tarefa neste trabalho será **alterar o caminho de dados** fornecido a **fim de incluir mais operações e módulos.** É necessário executar esse arquivo no **Google Colab**, sendo esta a plataforma que será utilizada para avaliar as submissões dos trabalhos.

- O trabalho pode ser feito em grupos compostos por **até 3 alunos.**

- Entregas após o prazo serão aceitas, porém haverá uma penalização. Quanto maior o atraso maior a penalização.

- Você deve entregar um único **arquivo zip,** contendo um arquivo **.ipynb** com a implementação do caminho de dados com as funções pedidas a seguir, em Verilog e um **relatório (descrito a seguir)**. Note que todas as funções devem estar no mesmo caminho de dados, ou seja, o trabalho é incremental, você deve entregar somente um caminho de dados contendo todas as funções solicitadas.

- Deverão ser implementados os arquivos Verilog e os códigos de teste em assembly das instruções. **As suas modificações devem ser feitas em uma cópia do código Verilog fornecido.**

- No mesmo arquivo zip contendo o caminho de dados, você deve e**nviar um relatório**, em pdf, explicando suas **decisões de projeto** e **testes realizados**. Este documento deve conter nome e matrícula de todos os integrantes do grupo.
    - Sugestão de tópicos:
        - Introdução: apresente os objetivos do trabalho, contextualize o tema e indique quais arquivos/funções foram alteradas.
        - Desenvolvimento: descreve a sua implementação separando as subseções para cada um dos problemas solicitados neste trabalho e descrevendo os testes implementados por vocês.
        - Conclusão: concluir o relatório fazendo um breve resumo do trabalho, as tomadas de decisão e os desafios durante a implementação

- **Cada grupo deve fazer somente uma submissão.**

2. Orientações para compilar os arquivos Verilog e rodar os testes

No .ipynb fornecido, você encontrará a implementação de um RISC-V monociclo com opções de visualização do caminho de dados.

- A implementação em Verilog do RISC-V está nas células "***Implementação do RISC-V***", essas são as células que precisam ser alteradas para incluir as novas operações.

- Para realizar um teste, deve-se informar o código de máquina em questão na célula ***"Cole o hex dump do seu código aqui".*** Utilize o ***Venus*** para obter o código (mais instruções na célula).

- Em "**Visualizar animação**" há uma versão interativa do estado do pipeline, onde você pode ver uma animação do estado do processador a cada ciclo de execução. Clique nos botões "**Previous**" e "**Next**" para visualizar o estado do sistema depois de cada instrução do código inserido.

3. Testes:

Para cada uma das instruções implementadas, inclua na documentação quais modificações foram feitas. Além disso, inclua um ou mais testes da instrução, utilizando as visualizações geradas pelo notebook para demonstrar o funcionamento.

4. Descrição da implementação:

Para este trabalho prático, deve-se implementar as seguintes instruções:

1. `add Rd, Rs1, Rs2`

```
Soma os valores dos registradores Rs1 e Rs2 e armazena o resultado em
Rd.

Exemplo: add x10, x15, x20 → [x10] = [x15] + [x20]
```

2. `rem Rd, Rs1, Rs2`

```
Calcula o resto da divisão de Rs1 por Rs2 e armazena o resultado em
Rd.

Exemplo: rem x10, x18, x5 → [x10] = [x18] % [x5]
```

3. `ori rd, rs1, imm`

```
Realiza a operação OR bit a bit entre o valor de Rs1 e o valor
imediato imm, e armazena o resultado em Rd.

Exemplo: ori x10, x2, 7 → [x5] = [x2] | 7
```

4. `beq Rs1, Rs2, label`

```
Se os valores de Rs1 e Rs2 forem iguais, desvia para o endereço
identificado por label.

Exemplo: beq x1, x2, loop → se [x1] == [x2], então PC = loop
```